

Resenha – Applying “Design by Contract” (Bertrand Meyer, 1992)

"Applying Design by Contract" é um artigo de Bertrand Meyer publicado na revista IEEE Computer em outubro de 1992. Meyer é o criador da linguagem de programação Eiffel e um dos nomes mais importantes da orientação a objetos — seu livro Object-Oriented Software Construction é referência obrigatória da área. O artigo em questão apresenta o conceito de Design by Contract (DbC), uma abordagem para especificação formal de componentes de software que o autor vinha desenvolvendo desde meados dos anos 80, e que ganhou notoriedade especialmente por ser a base do modelo de programação da linguagem Eiffel.

A ideia central do DbC parte de uma analogia simples com contratos comerciais. Em qualquer contrato entre duas partes, existem obrigações e benefícios para cada lado: o cliente se compromete a cumprir certas condições para receber um serviço, e o fornecedor garante entregar o resultado prometido caso essas condições sejam atendidas. Meyer transporta essa lógica para o software: toda rotina (método ou função) deve declarar explicitamente o que espera de quem a chama — a pré-condição — e o que garante entregar ao final da execução — a pós-condição. A essas duas assertivas se somam os invariantes de classe, que são propriedades que devem ser verdadeiras para qualquer instância de uma classe em qualquer momento em que ela esteja disponível para receber chamadas.

O artigo demonstra a aplicação prática do conceito com exemplos escritos em Eiffel, usando as cláusulas `requires` (pré-condição) e `ensures` (pós-condição) diretamente no código. Isso é um ponto importante: em Eiffel, os contratos não são apenas documentação — eles fazem parte da sintaxe da linguagem e podem ser verificados em tempo de execução durante o desenvolvimento, sendo desativados na versão final do produto para não prejudicar o desempenho. Meyer também discute a relação entre a força das assertivas e a distribuição de responsabilidade: quanto mais forte a pré-condição de uma rotina, mais o cliente precisa garantir antes de chamá-la; quanto mais forte a pós-condição, mais o fornecedor precisa entregar. Esse equilíbrio é deliberado e deve ser decidido no momento do design.

Um dos aspectos mais interessantes do texto é a forma como Meyer distingue o DbC da programação defensiva. Na programação defensiva, é o próprio método que verifica se as entradas são válidas e decide o que fazer quando não são. No DbC, essa responsabilidade é do cliente: se a pré-condição não foi satisfeita, o comportamento da rotina é indefinido — e não é obrigação do fornecedor tratar esse caso. Essa postura pode parecer rígida à primeira vista, mas Meyer argumenta que ela é mais honesta e mais fácil

de raciocinar sobre: a rotina só precisa funcionar corretamente dentro do domínio que ela declara aceitar, e quem a chama sabe exatamente o que precisa garantir.

A leitura do artigo levanta algumas questões que o texto não resolve completamente. A verificação em tempo de execução das assertivas é útil para encontrar bugs durante o desenvolvimento, mas ela não substitui uma prova formal de correção — e Meyer não entra a fundo nessa distinção. Além disso, a integração do DbC com linguagens que não têm suporte nativo a contratos, como Java ou C++, é tratada de forma bastante superficial. O autor menciona a possibilidade de simular as assertivas com condicionais e exceções, mas não discute as limitações dessa abordagem nem as soluções que surgiram depois, como bibliotecas de contrato ou anotações. Para um artigo de 1992 isso é compreensível, mas quem lê hoje sentirá falta dessa discussão.

Do ponto de vista do projeto de software, o DbC tem uma contribuição clara: ele força o designer a pensar nas responsabilidades de cada componente antes de escrever o código. As pré-condições e pós-condições são, na prática, uma especificação executável da interface — não apenas o que a rotina faz, mas sob quais condições ela funciona e o que ela garante. Isso se conecta diretamente a conceitos como separação de responsabilidades, encapsulamento e o princípio da substituição de Liskov, que exige que subclasses respeitem os contratos das classes que substituem.

No geral, é um artigo que envelheceu bem. A ideia continua relevante e influenciou ferramentas e práticas modernas — desde frameworks de teste baseados em propriedades até o uso de anotações de tipo em linguagens como Python e TypeScript, que podem ser vistas como uma forma mais fraca de especificação contratual. O texto é direto e bem exemplificado, embora o uso extensivo de Eiffel possa distanciar leitores menos familiarizados com a linguagem. Ainda assim, é uma leitura recomendada para quem quer entender de onde vem boa parte das ideias sobre especificação e correção de software que circulam na engenharia de software contemporânea.